实验报告——用pytorch实现CNN对cifar10数据集进行分类

概述

本次实验基于 PyTorch 实现 CNN 对 CIFAR-10 数据集的分类。考虑到 Colab 免费版硬件限制，选择轻量级 ResNet20 作为基础架构，通过调整网络结构与训练策略优化模型性能。

首先尝试原始 ResNet20，发现损失震荡、过拟合与收敛慢问题，随后逐步引入数据增强（随机翻转、裁剪、颜色扰动等）、学习率调度器（预热 + MultiStepLR）、标签平滑化等策略，并优化数据加载流程（用Kornia加速数据增强、调整 num_workers 等参数）以进一步缩短训练耗时。为进一步提升特征表达能力，最终在残差块中加入 SE 注意力机制，构建 SE-ResNet20。

经过多轮调参，模型在100个epoch后于测试集上达到 $92.6 \pm 0.2\%$ 的准确率，验证了轻量化设计与综合优化策略在解决本次任务上的有效性。

硬件和训练环境

我的模型在Google Colab上训练，用的是免费版，GPU是NVIDIA Tesla T4，分配的CPU有两个逻辑核。并把数据上传到Google Drive再装载到Colab来进行训练。

如果您要调试我的代码，可能需要对源代码中的 `datapath` 和 `writer_file` 变量进行修改，前者是训练数据的文件路径，后者是训练日志的输出路径。

另外，我不太确定我的代码在MLU上能否正常运行(在CPU上可以)，因为数据增强技术上涉及到了针对CUDA的优化。

最后在第三方库环境方面，除了 `torch` 和 `torchvision`，您可能还要检查 `tensorboard` 和 `kornia` 是否有安装。

```
pip install kornia
```

在Colab上训练时的云主机的一些环境版本参考：

```
Package                                  Version
---------------------------------------- -------------------
kornia                                   0.8.0
kornia_rs                                0.1.8
numpy                                    2.0.2
protobuf                                 5.29.4
tensorboard                              2.18.0
tensorflow                               2.18.0
torch                                    2.6.0+cu124
torchvision                              0.21.0+cu124
```

```
Python 3.11.12
```

网络结构的设计

使用Resnet20的缘由

由于硬件受限，平衡训练效率[1]和模型效能后，选择基于ResNet20来构造网络。

对于用CNN分类Cifar-10，ResNet是一个常用且有效的架构[2]。而ResNet20是ResNet[3]的一种变体，由于使用到的卷积核不大、整体网络不深，它非常的轻量，而且在Cifar-10这种图片尺寸比较小的数据集上表现不俗。[1:1]

ResNet20和较原始的ResNet（如ResNet50）不同的地方在于输入层少了一个MaxPool，而且不使用Bottleneck Block。

其浅层设计使Resnet20参数量更少（<1M参数），计算成本更低：

Layer	Input Shape	Unrolled Inputs	Unrolled Weights	Parameters	FLOPs	FLOPs (%)
Conv	[1, 3, 32, 32]	–	–	432	884,736	1.09
Block1-Conv1	[1, 16, 32, 32]	[1, 144, 1024]	[16, 144]	2,304	4,718,592	5.82
Block1-Conv2	[1, 16, 32, 32]	[1, 144, 1024]	[16, 144]	2,304	4,718,592	5.82
Block1-Conv3	[1, 16, 32, 32]	[1, 144, 1024]	[16, 144]	2,304	4,718,592	5.82
Block1-Conv4	[1, 16, 32, 32]	[1, 144, 1024]	[16, 144]	2,304	4,718,592	5.82
Block1-Conv5	[1, 16, 32, 32]	[1, 144, 1024]	[16, 144]	2,304	4,718,592	5.82
Block1-Conv6	[1, 16, 32, 32]	[1, 144, 1024]	[16, 144]	2,304	4,718,592	5.82
Block2-Conv1	[1, 16, 32, 32]	[1, 144, 256]	[32, 144]	4,608	2,359,296	2.91
Block2-Conv2	[1, 32, 16, 16]	[1, 288, 256]	[32, 288]	9,216	4,718,592	5.82
Block2-Conv3	[1, 32, 16, 16]	[1, 288, 256]	[32, 288]	9,216	4,718,592	5.82
Block2-Conv4	[1, 32, 16, 16]	[1, 288, 256]	[32, 288]	9,216	4,718,592	5.82
Block2-Conv5	[1, 32, 16, 16]	[1, 288, 256]	[32, 288]	9,216	4,718,592	5.82
Block2-Conv6	[1, 32, 16, 16]	[1, 288, 256]	[32, 288]	9,216	4,718,592	5.82
Block3-Conv1	[1, 32, 16, 16]	[1, 288, 64]	[64, 288]	18,432	2,359,296	2.91
Block3-Conv2	[1, 64, 8, 8]	[1, 576, 64]	[64, 576]	36,864	4,718,592	5.82
Block3-Conv3	[1, 64, 8, 8]	[1, 576, 64]	[64, 576]	36,864	4,718,592	5.82
Block3-Conv4	[1, 64, 8, 8]	[1, 576, 64]	[64, 576]	36,864	4,718,592	5.82
Block3-Conv5	[1, 64, 8, 8]	[1, 576, 64]	[64, 576]	36,864	4,718,592	5.82
Block3-Conv6	[1, 64, 8, 8]	[1, 576, 64]	[64, 576]	36,864	4,718,592	5.82
Linear	[1, 64]	–	–	650	1,300	0.00

例如下面显示的就是一个ResNet20结构：

Model Name	Dataset	N_{layers}	N_{stages}	$N_{filters}$	N_{params}
ResNet20	CIFAR10(100)	20	3	64,128,256	4.66M

不过我们模型使用的$N_{filters}$的具体值会有所调整。

网络结构设计

参考前人的经验[4][5]，针对Cifar-10设计具体的ResNet20模型：

```
# 整体模型设计
输入层(通道数 3->16)
    + Conv3x3 -> BN -> ReLU
中间层
    stage1(通道数 16->16)
        + 基础的ResidualBlock x3
    stage2(通道数 16->32)
        + 基础的ResidualBlock(stride=2 下采样)
        + 基础的ResidualBlock x2
    stage3(通道数 32->64)
        + 基础的ResidualBlock(stride=2 下采样)
        + 基础的ResidualBlock x2
输出层
    + GAP(全局平均池化) -> FC(64->10 全连接层)
```

每个ResidualBlock `Conv3x3 -> BN -> ReLU -> Conv3x3 -> BN` 都包含两个卷积层，把输出层的池化层也算进来，于是一共有$1+3*3*2+1=20$个卷积/池化层。

初始设置

我们起始的模型结构和超参数设置[1:2]：

激活函数	损失函数	优化器	epoch数	批大小
ReLU	CrossEntropyLoss	SGD（lr=0.1, momentum=0.9, weight_decay=5e-4）	160	128

另外还做了一些优化，希望能够加快训练过程[6][7]：

```
# 把num_workers设为非0值，加速data的加载
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True,num_workers=2)
```

训练日志部分使用 `tensorboard` 模块[8]，记录每个epoch训练集和测试集的准确率、训练集的平均损失。

网络参数调整

本节记录了完整的调参过程。

原始模型训练结果

发现:

- 训练集上的准确率远高于测试集，有过拟合问题
- 测试集准确率震荡严重
- 100 epoch之后收敛慢

调整一

进行了以下调整:

1. 加入基础**数据增强**（缓解过拟合 增加泛化性）[9]
2. 减少**epoch**数至100（减少训练耗时）
3. **训练日志** 新增对测试集的平均损失(AvgLoss_Test)的跟踪

```
#基础数据增强（随机水平翻转、随机裁剪+填充、数据归一化）
# 对Cifar-10进行统计后得到的参数，可以直接用于归一化
mean = [0.4914, 0.4822, 0.4465]
std = [0.2023, 0.1994, 0.2010]
Basictransform = transforms.Compose(
    [
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomCrop(32, padding=4, padding_mode='reflect'),
        transforms.ToTensor(),
        transforms.Normalize(mean,std)
    ]
)
```

调整后训练结果

发现:

- 训练集和测试集的准确率差距相较原始模型缩小，**过拟合有所缓解**
- 测试集的Acc、Loss**震荡**十分严重（猜测是后期**学习率过大**导致梯度下降失败）
- 100epoch时，训练集准确率相较原始模型更低了（猜测是数据增强导致**收敛变慢**）

调整二

1. 加入内存锁页、异步数据拷贝等机制，加速CPU→GPU的数据传输（希望加速训练过程）
2. 加入**学习率调度器**[1:3][10]（希望解决震荡问题）
3. 加入**Warmup** epochs[1:4]（主要想加速收敛）
4. 加入**标签平滑化**（正则化手段, 希望进一步缓解过拟合）
5. SGD优化器加入**Nesterov加速**（优化SGD，加速收敛），同时增加起始学习率至0.105（配合正则化手段）

```
# 调整DataLoader参数，加速CPU和GPU的数据对接
train_loader = DataLoader(
    ...,num_workers=4,pin_memory=True,prefetch_factor=3)

# 训练循环
for epoch in range(num_epochs):
    ...
    for i, (images, labels) in enumerate(train_loader):
        images = images.to(device,non_blocking=True) # 异步数据拷贝
        labels = labels.to(device,non_blocking=True)
```

```
# 学习率调度器(预热+StepLR)
warmup_epochs = 10
scheduler = SequentialLR(
        optimizer,
        schedulers=[
            LinearLR(optimizer, start_factor=0.01, total_iters=warmup_epochs),  # 0.01→0.1
            MultiStepLR(optimizer, milestones=[40,80], gamma=0.1)
        ],
        milestones=[warmup_epochs]
    )
```

```
# 标签平滑化
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
# 调整优化器
optimizer = optim.SGD(model.parameters(),lr=0.105, momentum=0.9, weight_decay=5e-4,nesterov=True)
```

调整后训练结果

发现:

- 训练时长有效缩减（57min -> 50min）
- 测试集的震荡问题得到解决（随着两次学习率调整）
- 整体准确率很大提升（训练集>96%, 测试集>92%），收敛得很好
- 训练集和测试集的准确率差距>5%，**过拟合**问题有缓解但**仍存在**

调整三

进一步强化**数据增强**技术（加入色彩变换和随机擦除）

```
Basictransform = transforms.Compose(
    [
        ...,
        transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
        ...,
        transforms.RandomErasing(p=0.5, scale=(0.02, 0.33), ratio=(0.3, 3.3)),
        ...
    ]
)
```

调整后训练结果

发现:

- 训练集和测试集的准确率差距在1%左右，在没有降低测试集准确率的基础上，模型**泛化性得到显著提升**
- **训练时长**变长（50min -> 1.3h）
- 似乎还有能**继续收敛**的趋势

调整四

1. 增加epoch数至160(希望模型能继续收敛)
2. 配合epoch数调整学习率调度器
3. 利用Kornia库，使得数据增强技术部分的图像处理能在gpu上进行[11][12] （提高gpu利用率，加快训练速度）

```
# epoch数调整
epoch = 160
warmup_epochs = 15

# 学习率调度器调整
scheduler = SequentialLR(
        ...
        schedulers=[
            ...
            MultiStepLR(optimizer, milestones=[50,80,120], gamma=0.1)
        ],
        milestones=[warmup_epochs]
    )
```

```
# 数据增强
# 改用K.AugmentationSequential(原本是torchvision的transforms.Compose)
Basictransform = transforms.Compose(
    [
        transforms.ToTensor()
    ]
)
train_dataset = datasets.CIFAR10(root=datapath, train=True, download=True, transform=Basictransfor
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True,num_workers=4,pin_mem

gpu_augment = K.AugmentationSequential(
    K.RandomHorizontalFlip(p=0.5),
    K.RandomCrop((32, 32),padding=4,padding_mode='reflect'),
    K.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, p=1.0),
    K.RandomErasing(p=0.5, scale=(0.02, 0.33), ratio=(0.3, 3.3)),
    K.Normalize(torch.tensor(mean), torch.tensor(std))
).to(device)

...

# 训练循环
for epoch in range(num_epochs):
    model.train()
    epoch_loss = 0.0
    epoch_corrects = 0
    for i, (images, labels) in enumerate(train_loader):

        images = images.to(device,non_blocking=True)
        labels = labels.to(device,non_blocking=True)

        # GPU上执行增强（自动批处理）
        with torch.no_grad():
            images = gpu_augment(images)

        outputs = model(images)
        loss = criterion(outputs, labels)
        ...
```

调整后训练结果

发现:

- 模型性能相较调整三没有明显的提升
- **过拟合**问题重新出现
- 训练的**时间效率**得到显著提升（1.3h可以跑160个epoch）

调整五

1. 调整Resnet20架构，加入注意力机制（加入SE块）
2. 为节省时间成本，设置以下参数
   `epoch=100`、`warmup_epochs=10`、`MultiStepLR(...milestones=[40,80]...)`

SE-Resnet是Resnet[13]的一种拓展，在Resnet的基础上加入了SE块，为模型加入了注意力机制，使得模型获得channel-wise attention，能够补充一定的全局信息。

我们这里使用了In-block POST SE-Resnet的设计，这个设计会在每个残差块的卷积之后、跳跃连接之前插入一个SE块
[14]：`Conv3x3 -> BN -> ReLU -> Conv3x3 -> BN -> SE -> 与identity跳跃链接`。

```python
# 定义SE块
class SEBlock(nn.Module):
    def __init__(self, in_channels, reduction_ratio=8):
        super(SEBlock, self).__init__()
        self.reduction_ratio = reduction_ratio

        # Squeeze
        self.squeeze = nn.AdaptiveAvgPool2d(1)

        # Excitation
        self.excitation = nn.Sequential(
            nn.Linear(in_channels, in_channels // reduction_ratio, bias=False),
            nn.Mish(inplace=True),  # 使用Mish激活函数
            nn.Linear(in_channels // reduction_ratio, in_channels, bias=False),
            nn.Sigmoid()
        )

        # kaiming初始化权重 针对Mish
        # 由于nonlinearity没有mish 所以要手动推导等效增益
        gain = torch.nn.init.calculate_gain('leaky_relu', 0.1)
        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.kaiming_normal_(m.weight, mode='fan_in', nonlinearity='leaky_relu')
                with torch.no_grad():
                    m.weight.data *= gain

    def forward(self, x):
        batch_size, c, _, _ = x.size()
        # Squeeze
        squeezed = self.squeeze(x).view(batch_size, c)
        # Excitation
        weights = self.excitation(squeezed).view(batch_size, c, 1, 1)
        # Scale
        return x * weights.expand_as(x)
```

```python
# 定义SE残差块(In-block POST SE-Res设计)
class SEResidualBlock(nn.Module):
    def __init__(self, inchannel, outchannel, stride=1):
        super(SEResidualBlock, self).__init__()
        ...
        self.se = SEBlock(outchannel,8)
        ...
    def forward(self, identity):
        out = self.residual(identity)
        out = self.se(out) # SE块的输出与输入向量的shape一致
        out += self.shortcut(identity)
        out = F.relu(out)
        return out
```

调整后训练结果

我们把它在最后10个epoch的表现和调整三的做比较：

发现:

- 改用SE-Resnet后模型似乎收敛得更快（最后10 epoch的测试集准确率的范围从 $[0.918, 0.926] \rightarrow [0.925, 0.929]$）

关于数据增强技术

数据增强技术的使用，总结而言会对模型产生两点影响：

- 强化模型泛化能力
- 会降低训练速度（需要进行额外的数据处理）

一些遇到的挑战与解决办法

环境问题
一开始撰写完python代码的时候，打算在本地调试一下，结果调用Tensorboard时发现我的conda环境出了兼容性问题（报错protobuf版本不兼容）。

随后用互联网和大模型搜索，确认了配环境需要特别注意Tensorflow和pytorch对python、

numpy、protobuf等模块的版本支持，最后重新建立conda环境并统一用pip下载库，解决了问题。

AMP加速尝试失败

其实在调整一的时候尝试了加入**AMP**加速训练[15]，期望减少训练耗时，但结果发现准确率曲线震荡更严重了，当时以为是AMP导致了一些梯度传播的问题。

后面透过**消融实验**确认了AMP不是主因，但由于**提速不明显**，而且我对这个技术还不够熟悉，不知道如何控制潜在的负面影响（例如降低准确率），所以最终还是取消了这个优化行为。

```
# AMP加速
scaler = GradScaler("cuda",enabled=use_AMP)
for epoch in range(num_epochs):
    model.train()
    for i, (images, labels) in enumerate(train_loader):
        images = images.to(device,non_blocking=True)
        labels = labels.to(device,non_blocking=True)

        optimizer.zero_grad()
        with autocast(device_type="cuda",enabled=use_AMP):
            outputs = model(images)
            loss = criterion(outputs, labels)

        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
```

最终模型性能

最终表现最优的结构和参数就是调整五后的模型！（源代码中用的就是这个）

与报告开头的初始设置相比，主要的改动：

1. 采用了SE-Resnet结构（加入注意力机制）
2. 加入了数据增强技术
3. 优化了训练过程的硬件利用
    - 用`Kornia`库把数据增强的任务移动到GPU进行
    - 调整`num_workers`、`pin_memory`、`prefetch_factor`、`non_blocking`参数来加速CPU到

GPU的数据传输

4. 加入标签平滑化（正则化手段）
5. 加入了学习率调度器(预热+StepLR)
6. SGD优化器加入Nesterov加速

在训练集上的准确率：93.31%
在测试集上的准确率：92.85%

---

1. 语言大模型辅助全过程
2. ResNet on Cifar-10
3. 什么是残差网络
4. ResNet20实现1
5. ResNet20实现2
6. num workers 设置
7. num workers 设置
8. 训练日志使用
9. 数据增强使用启发和引导(大语言模型)
10. 学习率调度器
11. Kornia使用启发和引导(大语言模型)
12. Kornia官方文档:augmentation
13. Resnet family之SE-Resnet
14. SE-Resnet使用启发和引导(大语言模型)
15. AMP加速